

Chapter 5: The Vehicle Routing Problem with Time Windows

Vehicle Routing: Problems, Methods, and Applications (2nd ed., 2014)

Guy Desaulniers Oli B.G. Madsen Stefan Ropke

Based on Chapter 5, pp. 119–171

Slides prepared for self-study

Chapter Outline

- 1 Introduction
- 2 Mathematical Formulations
- 3 Exact Solution Methods
- 4 Heuristics
- 5 Extensions
- 6 Conclusions and Future Research Directions
- 7 Bibliography

What is the VRPTW? I

The **Vehicle Routing Problem with Time Windows (VRPTW)** is the most widely studied extension of the classical Capacitated VRP (CVRP). It arises whenever each customer must be visited within a prescribed time interval, called a *time window* $[e_i, l_i]$.

A **concrete real-world example** illustrates the setting clearly. Consider a city logistics company that delivers parcels to n retail stores. The company depot opens at 07:00 and closes at 18:00. Each store has a contractual delivery window—for instance, a supermarket might require delivery between 08:00 and 10:00 to stock shelves before opening. The company must design a set of vehicle routes that together visit all stores exactly once, ensure that each store is reached within its window, and minimize the total travel distance (or the fleet size, or both).

More precisely, the VRPTW is defined on a complete directed graph $G = (V, A)$ where $V = \{0, 1, \dots, n\}$ is the vertex set and A is the arc set. Vertex 0 represents the depot and vertices $1, \dots, n$ represent the n customers. Each arc $(i, j) \in A$ carries a **travel time** t_{ij} and a **travel cost** c_{ij} . Each customer $i \geq 1$ has a **demand** $q_i > 0$, a **service time** $s_i \geq 0$ (the time to unload goods at the customer's site), and a **time window** $[e_i, l_i]$ specifying the earliest and latest times at which service may begin. A **fleet** of m identical vehicles, each with capacity Q , is stationed at the depot.

What is the VRPTW? II

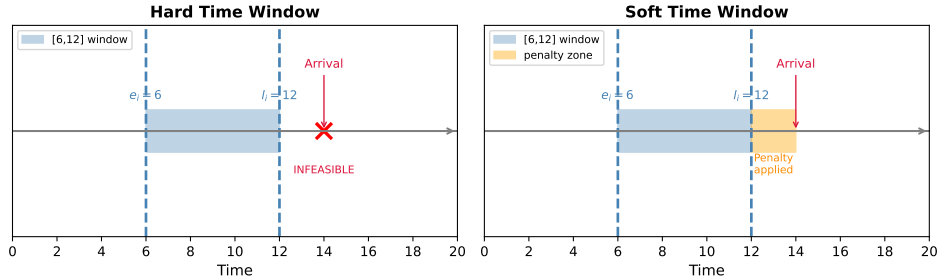
A vehicle that arrives at customer i *before* e_i must wait until e_i before starting service. Whether a vehicle that arrives *after* l_i is allowed depends on whether time windows are **hard** or **soft**:

- **Hard time windows:** arriving after l_i is strictly infeasible—the constraint $t_{\text{arrival}} \leq l_i$ must hold.
- **Soft time windows:** late arrival is permitted but incurs a penalty proportional to the tardiness $\max(0, t_{\text{arrival}} - l_i)$.

The figure below shows both types for a window $[6, 12]$ when a vehicle arrives at time 14.

What is the VRPTW? III

Hard vs. Soft Time Windows



What is the VRPTW? IV

Historical context. The VRPTW was introduced independently by Solomon [1] and by Desrochers, Desrosiers, and Solomon [2] in the late 1980s. Solomon also introduced the benchmark instances that remain the standard testbed to this day. Since then, hundreds of papers have appeared, spanning exact methods, construction heuristics, local search, and population-based metaheuristics.

Why is VRPTW harder than CVRP? Time windows shrink the feasible region: a route must not only be capacity-feasible but also time-feasible. Even checking whether a given sequence of customers is feasible requires propagating departure times forward (and sometimes backward) along the route. This *temporal propagation* makes neighbourhood evaluation more expensive and adds combinatorial structure that exact methods must exploit.

Objectives. In practice, two objectives are common and they are typically lexicographically ordered:

- 1 Minimize the **number of vehicles** used (fleet size is a fixed cost).
- 2 Subject to (1), minimize the **total travel distance** (or time).

What is the VRPTW? V

Some models only minimize total distance for a fixed fleet.

Key Insight

The VRPTW is $\mathcal{NP}$ -hard because it contains the CVRP as a special case (set all time windows to $[0, \infty)$). In practice, tight time windows can paradoxically make instances *easier* for exact methods by pruning large parts of the search space.

Network Flow Formulation (Model I) I

The most natural formulation uses **binary arc variables** that track whether each vehicle travels each arc, together with **continuous time variables** that record when each vehicle arrives at each customer.

Let the vertex set be $V = \{0, 1, \dots, n+1\}$ where vertex 0 is the origin depot and vertex $n+1$ is a copy of the depot representing the destination. The fleet has m vehicles indexed $k = 1, \dots, m$. Let K denote the set of vehicles.

Decision Variables

- $x_{ijk} \in \{0, 1\}$: equals 1 if vehicle k traverses arc (i, j) , and 0 otherwise.
- $T_{ik} \geq 0$: the time at which vehicle k *begins service* at customer i (or departs the depot for $i = 0$).

Network Flow Formulation (Model I) II

VRPTW Model I (three-index formulation)

$$\text{minimize} \quad \sum_{k \in K} \sum_{(i,j) \in A} c_{ij} x_{ijk} \quad (5.1)$$

$$\text{subject to} \quad \sum_{k \in K} \sum_{j \in V} x_{ijk} = 1, \quad \forall i \in N, \quad (5.2)$$

$$\sum_{j \in V} x_{0jk} = 1, \quad \forall k \in K, \quad (5.3)$$

$$\sum_{i \in V} x_{ihk} - \sum_{j \in V} x_{hjk} = 0, \quad \forall h \in N, k \in K, \quad (5.4)$$

$$\sum_{i \in V} x_{i,n+1,k} = 1, \quad \forall k \in K, \quad (5.5)$$

$$\sum_{i \in N} q_i \sum_{j \in V} x_{ijk} \leq Q, \quad \forall k \in K, \quad (5.6)$$

$$T_{ik} + s_i + t_{ij} - T_{jk} \leq M(1 - x_{ijk}), \quad \forall (i,j) \in A, k \in K, \quad (5.7)$$

$$s_i \leq T_{ik} \leq l_i \quad \forall i \in V, k \in K \quad (5.8)$$

Set-Partitioning Formulation (Model II) I

A computationally far more powerful approach is to define a variable for each *complete feasible route* and reformulate the VRPTW as a **set-partitioning problem**.

Let Ω be the set of all feasible routes (i.e., sequences of customers starting and ending at the depot that respect capacity and time windows). Each route $p \in \Omega$ has an associated **cost** c_p (total travel distance along the route) and a binary coefficient $a_{ip} \in \{0, 1\}$ indicating whether customer i is visited on route p .

Decision Variable

$\lambda_p \in \{0, 1\}$: equals 1 if route p is used in the solution, and 0 otherwise.

Set-Partitioning Formulation (Model II) II

VRPTW Model II (set-partitioning formulation)

$$\text{minimize } \sum_{p \in \Omega} c_p \lambda_p \quad (5.13)$$

$$\text{subject to } \sum_{p \in \Omega} a_{ip} \lambda_p = 1, \quad \forall i \in N, \quad (5.14)$$

$$\sum_{p \in \Omega} \lambda_p \leq m, \quad (5.15)$$

$$\lambda_p \in \{0, 1\}, \quad \forall p \in \Omega. \quad (5.16)$$

Set-Partitioning Formulation (Model II) III

Interpretation. The objective (5.13) minimizes total travel cost. Constraint (5.14) ensures each customer i appears on exactly one selected route—this is the “set partitioning” condition (using ≥ 1 instead would yield set covering, which allows redundant visits). Constraint (5.15) limits the total number of routes to the fleet size m .

Exponential column set. The set Ω is exponentially large (it contains all feasible routes over all possible orderings of all subsets of N). Therefore, Model II cannot be solved by enumerating all columns. Instead, the LP relaxation of Model II is solved by **column generation**, which generates only the columns with the potential to improve the objective—as detailed in Section 3.

Relationship between Model I and Model II. Both formulations describe the same problem; Model II typically gives a tighter LP relaxation because feasible routes implicitly encode all time-window and capacity constraints. The LP bound from Model II therefore lies at or above the Model I LP bound.

Set-Partitioning Formulation (Model II) IV

Key Insight

The set-partitioning formulation is the foundation of the best-performing exact algorithms for the VRPTW. Column generation efficiently identifies which routes to include, while branch-and-bound drives the solution toward integrality.

Soft Time Windows and Service Time I

In the **soft time window** variant (SVRPTW), each customer i has a preferred window $[e_i, l_i]$ but may be visited outside it at a penalty. Two common penalty structures exist:

Linear Penalty Function

The penalty for serving customer i at time T_i is:

$$p_i(T_i) = \alpha_i \cdot \max(0, e_i - T_i) + \beta_i \cdot \max(0, T_i - l_i),$$

where $\alpha_i \geq 0$ is the *early penalty* rate (e.g., cost of waiting, tying up a loading dock) and $\beta_i \geq 0$ is the *late penalty* rate. Early arrival costs are sometimes zero (the vehicle simply waits), making $\alpha_i = 0$ common. Late arrival penalties model contractual fines.

Service time. At each customer i , once the vehicle arrives and the time window opens, the driver spends s_i time units serving the customer (unloading, obtaining signatures, etc.). The vehicle departs at time $T_i + s_i$. Therefore the *earliest possible departure* is $e_i + s_i$ and the *latest permissible arrival* for a hard window is l_i .

Soft Time Windows and Service Time II

Waiting time. If a vehicle arrives at customer i at time $a_i < e_i$, it must wait $w_i = e_i - a_i$ before starting service. The total time spent at customer i (waiting + service) is therefore $w_i + s_i$. For hard windows, waiting is free but unavoidable; for soft windows, arriving too early may also incur a penalty if $\alpha_i > 0$.

Compatibility with Model I. In Model I, adding soft windows simply replaces constraints (5.8) with bounds on T_{ik} , and adds penalty terms to the objective. The structure of the problem is unchanged; only the feasible region is enlarged.

Overview of Exact Methods I

Exact algorithms guarantee finding the globally optimal solution (or proving infeasibility). For the VRPTW, three families of exact methods have been studied:

- 1 **Branch-and-Bound (B&B)** applied directly to Model I (three-index MIP): practical only for small instances ($n \leq 25$).
- 2 **Branch-and-Price (B&P)**: combines column generation (to solve LP relaxations of Model II) with branch-and-bound (to recover integer solutions). This is the dominant exact approach.
- 3 **Branch-and-Price-and-Cut (B&P&C)**: augments B&P with cutting planes to strengthen bounds.

The critical challenge is solving the **pricing subproblem**: at each node of the B&P tree, one must find the route with the most negative *reduced cost* with respect to the current dual variables. This is an **Elementary Shortest Path Problem with Resource Constraints (ESPPRC)**, which is itself $\mathcal{NP}$ -hard.

Overview of Exact Methods II

Complexity context. The LP relaxation of Model II has $|\Omega|$ variables (exponentially many) and $n + 1$ constraints (one per customer plus the fleet bound). By LP duality, the dual of this LP has $n + 1$ variables, and the dual constraint for each route p reads:

$$c_p - \sum_{i \in N} a_{ip} \pi_i - \mu \geq 0,$$

where π_i is the dual variable (shadow price) for covering customer i and μ is the dual variable for the fleet constraint. Checking optimality requires verifying that no route $p \in \Omega$ violates this constraint, i.e., no route has a *reduced cost* $\bar{c}_p = c_p - \sum_i a_{ip} \pi_i - \mu < 0$. Finding such a route (if one exists) is exactly the pricing subproblem.

Column Generation for the VRPTW LP I

Column generation is an iterative procedure for solving the LP relaxation of the set-partitioning model without enumerating all columns. At each iteration, only a subset of routes (columns) is maintained in the **Restricted Master Problem (RMP)**.

Column Generation for the VRPTW LP II

Column Generation for VRPTW LP Relaxation

Restricted Master Problem (RMP)

Minimize: $\sum_p c_p \lambda_p$
s.t. cover each customer ≥ 1
Solve LP $\Rightarrow$ dual prices π_i
Variables: route columns λ_p

dual prices π

new column

Pricing Subproblem (ESPPRC)

Find route with $\bar{c} < 0$:

$$\bar{c} = c_p - \sum_i a_{ip} \pi_i$$

Shortest path with resource
constraints on $G = (V, A)$

Repeat until no column with negative reduced cost exists $\Rightarrow$ LP optimal

Column Generation for the VRPTW LP III

The algorithm works as follows. Start with an initial set of routes (e.g., one route per customer). Solve the LP relaxation of the RMP to obtain primal variables λ_p and dual variables π_i . Pass the dual prices to the **pricing subproblem**, which searches for a route p with $\bar{c}_p < 0$. If such a route is found, add it to the RMP and re-solve. If no such route exists, the current LP solution is **optimal** for the full set-partitioning LP relaxation.

Column Generation for the VRPTW LP IV

Column Generation Procedure

Input: graph G , dual prices π_i , fleet dual μ .

Output: optimal LP solution to set-partitioning relaxation.

- 1 Initialise RMP with a set of artificially-expensive routes (e.g., one giant route per customer).
- 2 **Repeat:**
 - a Solve LP relaxation of RMP $\rightarrow$ primal λ^* , duals π^* , μ^* .
 - b Solve pricing subproblem: find $p^* = \arg \min_{p \in \Omega} \bar{c}_p = c_p - \sum_i a_{ip} \pi_i^* - \mu^*$.
 - c If $\bar{c}_{p^*} < 0$: add column p^* to RMP, go to (a).
 - d Else: **stop** — LP optimal.

Key insight about dual prices. The dual variable π_i represents the marginal cost of covering customer i one more time. A route with negative reduced cost “earns more” (in terms of reduced cost) than it costs: adding it to the basis improves the objective.

Convergence. Column generation is guaranteed to terminate in finitely many iterations because $|\Omega|$ is finite. In practice, convergence can be slow due to *tailing-off*—many iterations with tiny improvements. Stabilisation techniques (dual variable smoothing, interior-point methods) mitigate this.

The Pricing Subproblem: ESPPRC I

The pricing subproblem for the VRPTW is a **Shortest Path Problem with Resource Constraints (SPPRC)** on the network $G = (V, A)$. Each arc $(i, j) \in A$ has a **reduced cost** $\bar{c}_{ij} = c_{ij} - \pi_j$ (incorporating the dual price of visiting customer j). The goal is to find the least-cost path from the depot 0 to the destination $n + 1$ that is both *capacity-feasible* and *time-feasible*.

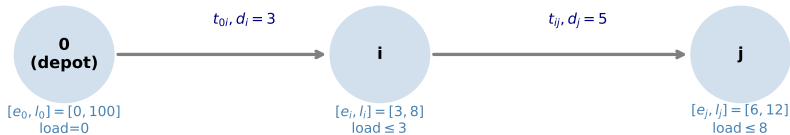
Resources. Each partial path carries two resources:

- **Time resource** T : the current time (including service and waiting).
- **Load resource** W : total demand accumulated so far.

The Pricing Subproblem: ESPPRC II

SPPRC: Resource Extension Along a Path

Extension: $T_j = \max(e_j, T_i + s_i + t_{ij})$, $Q_j = Q_i + q_j$



The Pricing Subproblem: ESPPRC III

Resource extension function. When extending a partial path ending at node i by arc (i,j) , the resources are updated as:

$$T_j = \max(e_j, T_i + s_i + t_{ij}), \quad (\text{REF-T})$$

$$W_j = W_i + q_j. \quad (\text{REF-W})$$

The max in (REF-T) models waiting at j if the vehicle arrives before e_j . A partial path is **feasible** if $T_j \leq l_j$ and $W_j \leq Q$ at every node j visited.

The Pricing Subproblem: ESPPRC IV

Label-Setting Algorithm for SPPRC

Each partial path is represented by a *label* $L = (v, T, W, \bar{c})$ recording the current node v , time T , load W , and accumulated reduced cost $\bar{c}$.

- 1 Initialise: label $L_0 = (0, e_0, 0, 0)$ at depot.
- 2 Select unprocessed label $L = (v, T, W, \bar{c})$ with smallest $\bar{c}$ (Dijkstra-like if costs are non-negative).
- 3 For each arc $(v, j) \in A$: extend L to $L' = (j, T_j, W_j, \bar{c} + \bar{c}_{vj})$. Check feasibility: if $T_j \leq l_j$ and $W_j \leq Q$, add L' to unprocessed set.
- 4 **Dominance pruning:** label L_1 *dominates* L_2 at the same node if $\bar{c}_1 \leq \bar{c}_2$, $T_1 \leq T_2$, $W_1 \leq W_2$ (and at least one is strict). Discard L_2 .
- 5 Repeat until no unprocessed labels remain.
- 6 Return the label at $n + 1$ with minimum $\bar{c}$.

The Pricing Subproblem: ESPPRC V

Elementarity. The standard SPPRC allows a customer to be visited more than once (cycles in the path), which can produce solutions that are infeasible for the VRPTW (a customer cannot be served twice). The **Elementary SPPRC (ESPPRC)** adds the constraint that each customer is visited *at most once*. This makes the problem $\mathcal{NP}$ -hard.

In the label algorithm, elementarity is enforced by augmenting each label with a **visited set** S : $L = (v, T, W, \bar{c}, S)$, where $S \subseteq N$ records which customers have been visited. When extending to node j , the extension is allowed only if $j \notin S$. The dominance rule must also compare visited sets.

Complexity trade-off. Tracking the visited set S requires 2^n bits per label in the worst case, making the ESPPRC exponential. In practice, for small n or with bitmask representations, ESPPRC is tractable. For larger instances, relaxations of elementarity (*ng-routes*, *subset-row cuts*) balance tractability and LP-bound quality.

The Pricing Subproblem: ESPPRC VI

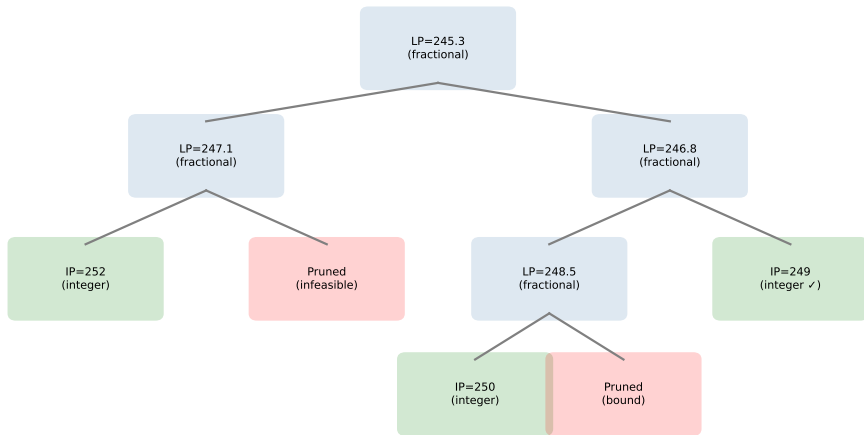
Key Insight

The SPPRC pricing algorithm is the computational bottleneck of branch-and-price for VRPTW. Improving the pricing step via better dominance rules, bidirectional labelling, and ng-relaxation is the most impactful avenue for exact algorithm research.

Branch-and-price integrates column generation into a branch-and-bound framework. At each node of the B&B tree, the LP relaxation of the current RMP is solved by column generation. When the LP solution is integer, a feasible solution is found. When it is fractional, branching is applied to drive integrality.

Branch-and-Price for VRPTW II

Branch-and-Price Search Tree for VRPTW
(LP = LP relaxation bound, IP = integer solution)



Branch-and-Price for VRPTW III

Branching strategies. Standard LP branching (branch on a fractional λ_p) is *problematic* because fixing $\lambda_p = 0$ or $\lambda_p = 1$ disrupts the structure of the pricing subproblem. Instead, two branching strategies are commonly used:

- ➊ **Arc-based branching (Ryan-Foster):** choose a pair (i, j) of customers such that they are sometimes on the same route and sometimes not. Branch on “ i and j are always on the same route” versus “ i and j are never on the same route.” This corresponds to adding arcs or forbidding them in the pricing graph, which is compatible with the ESPPRC.
- ➋ **Vehicle-number branching:** branch on the number of vehicles used (fix the fleet constraint $\sum_p \lambda_p \leq m$ from above and below).

Branch-and-Price for VRPTW IV

Bounding and pruning. The LP lower bound at each B&B node is the value of the RMP LP relaxation. A node is *pruned* if:

- its LP bound is $\geq$ the best integer solution found so far (bound pruning);
- its LP is infeasible (infeasibility pruning);
- its LP solution is integer (solution found).

Strengthening with cuts. **Branch-and-Price-and-Cut** adds cutting planes to the RMP to tighten LP bounds. The most effective cuts for VRPTW are *subset-row inequalities* (generalised subtour elimination) and *capacity cuts*. These cuts are added as constraints to the RMP but complicate the pricing subproblem because they introduce additional dual variables.

State of the art. Desaulniers, Desrosiers, and Solomon's branch-and-price framework from the 1990s could solve Solomon's 100-customer instances to optimality. More recent algorithms (Baldacci, Mingozzi, Roberti 2012; Costa, Contardo, Desaulniers 2019) use ng-route relaxation and aggressive cutting to solve instances with up to 400 customers.

Branch-and-Price for VRPTW V

Key Insight

Branch-and-price is superior to direct MIP solving for VRPTW because the set-partitioning LP provides much tighter lower bounds than the arc-flow LP, making the B&B tree much smaller.

Dual Variable Stabilisation I

A practical challenge in column generation is **instability of dual variables**: between iterations, the dual solution π^* can oscillate wildly, especially near optimality. This is the *tailing-off* or *degeneracy* problem.

Stabilisation approaches keep the dual variables close to a reference point $\hat{\pi}$ by adding a penalty for moving too far:

Proximal Stabilisation (BoxStep)

Instead of solving the RMP directly, solve:

$$\min_y \{c^T y : Ay = b, \hat{\pi} - \delta \leq \pi(y) \leq \hat{\pi} + \delta\}$$

where $\delta > 0$ is a step-size parameter and $\pi(y)$ denotes the dual variables of the RMP. This constrains duals to a “box” around the reference point $\hat{\pi}$, dampening oscillations.

Stabilisation reduces the number of column generation iterations (and hence the number of ESPPRC pricing calls) substantially—often by a factor of 2–5—leading to significant speed-ups on large instances.

Benchmark Results for Exact Methods I

Instance	Class	n	BKS (NV)	BKS (TD)	Solved?
C101	C1	100	10	828.94	Yes
C201	C2	100	3	591.56	Yes
R101	R1	100	20	1645.79	Yes
R201	R2	100	4	1252.37	Yes
RC101	RC1	100	14	1696.95	Yes
RC201	RC2	100	4	1406.94	Yes

NV = Number of vehicles, TD = Total distance, BKS = Best known solution.

All 56 Solomon 100-customer instances have been solved to optimality. The C-class (clustered customers) instances are typically solved fastest because their tight windows allow aggressive label dominance pruning in the ESPPRC. R-class instances (random customers, tight windows) are the hardest.

Larger benchmark sets (200, 400, 1000 customers) remain open: no algorithm has solved all 400-customer Gehring-Homberger instances to proven optimality.

Overview of Heuristic Methods I

Because the VRPTW is $\mathcal{NP}$ -hard, exact methods are impractical for large real-world instances.

Heuristics trade optimality guarantees for speed. The literature organises VRPTW heuristics into three tiers:

- ① **Construction heuristics:** build an initial feasible solution from scratch in polynomial time. Fast but produces mediocre quality.
- ② **Local search (improvement heuristics):** start from a feasible solution and iteratively apply neighbourhood moves to improve it. Usually combined with construction.
- ③ **Metaheuristics:** systematic frameworks (tabu search, simulated annealing, genetic algorithms, large neighbourhood search) that guide local search to escape local optima.

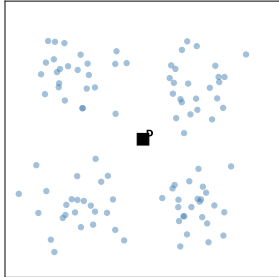
The **standard testbed** is Solomon's six instance classes with $n = 100$ customers each, organised as follows:

- **C1, C2:** customers arranged in clusters; C1 has tight time windows, C2 has wide windows.
- **R1, R2:** customers placed uniformly at random; R1 tight, R2 wide.
- **RC1, RC2:** mixed random and clustered; RC1 tight, RC2 wide.

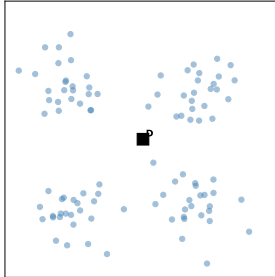
Overview of Heuristic Methods II

Solomon Benchmark Instance Classes (100 customers each)

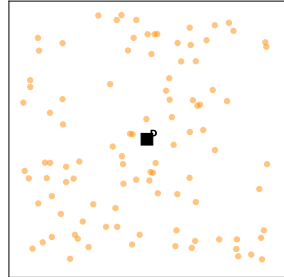
**C1: Clustered
(tight windows)**



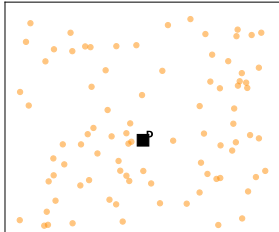
**C2: Clustered
(wide windows)**



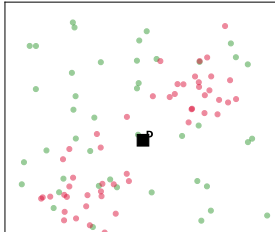
**R1: Random
(tight windows)**



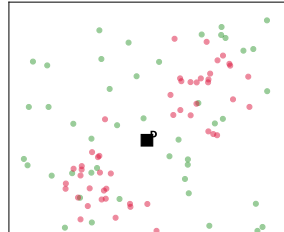
**R2: Random
(wide windows)**



**RC1: Random+Cluster
(tight windows)**



**RC2: Random+Cluster
(wide windows)**



Construction Heuristic: Solomon's I1 Algorithm I

Solomon's **I1 (cheapest insertion)** algorithm is the most widely used construction heuristic for the VRPTW. The idea is to build routes one at a time: repeatedly select the unrouted customer whose insertion into the current route is cheapest (in a combined cost that balances travel distance and time-window tightness).

Intuition. Rather than inserting customers in an arbitrary order, I1 uses a *combined criterion* $c_2(i, u, j)$ that measures the *desirability* of inserting customer u between consecutive customers i and j on the current route. Customers that are close (small detour) and fit well into the schedule (small time push) score highly.

Construction Heuristic: Solomon's I1 Algorithm II

Solomon I1 Algorithm

Input: depot 0, unrouted customers U , current route $\rho = (r_1, r_2, \dots, r_k)$ (initialised with depot).

① **Seed:** choose the unrouted customer farthest from the depot as the first customer on a new route.

② **While** $U \neq \emptyset$:

ⓐ For each $u \in U$ and each feasible insertion position (r_m, r_{m+1}) in ρ :

$$c_1(r_m, u, r_{m+1}) = \alpha_1 d_1(r_m, u, r_{m+1}) + \alpha_2 d_2(r_m, u, r_{m+1})$$

where d_1 is the extra distance (detour) and d_2 is the time delay pushed to r_{m+1} and beyond.

ⓑ For each u , let $c_1^*(u) = \min$ over positions.

ⓒ Compute $c_2(u) = \lambda \cdot d(0, u) - c_1^*(u)$ (the *benefit* of inserting u now vs. later).

ⓓ Insert the $u^* = \arg \max_u c_2(u)$ at its best position. If no feasible insertion: open a new route for u^* .

Construction Heuristic: Solomon's I1 Algorithm III

Worked Example: Solomon I1 with 5 Customers

Consider depot $D = (5, 5)$ and 5 customers with positions and time windows: $c_1 = (2, 8, [1, 5])$, $c_2 = (4, 9, [2, 8])$, $c_3 = (9, 4, [1, 6])$, $c_4 = (2, 2, [3, 9])$, $c_5 = (7, 2, [2, 7])$. Vehicle capacity $Q = 20$, all demands $q_i = 4$.

Step 1 (seed): distances from depot: $d(D, c_1) = 4.24$, $d(D, c_2) = 4.47$, $d(D, c_3) = 5.66$, $d(D, c_4) = 4.24$, $d(D, c_5) = 3.61$. Farthest: $c_3 \rightarrow$ route $\rho = (D, c_3, D)$.

Step 2a: try inserting c_1 between (D, c_3) :

$d_1 = d(D, c_1) + d(c_1, c_3) - d(D, c_3) = 4.24 + 7.62 - 5.66 = 6.20$. Time push: arrival at c_3 shifts from 5.66 to $4.24 + s_1 + 2.00 + s_3$. Check feasibility: arrival at

$c_3 = 4.24 + 0 + \sqrt{(9-2)^2 + (4-8)^2} = 4.24 + 8.06 = 12.30 > l_3 = 6$. Infeasible!

Step 2a (cont.): insert c_2 : $d(D, c_2) = 4.47$, $d(c_2, c_3) = \sqrt{25 + 25} = 7.07$. Detour $d_1 = 4.47 + 7.07 - 5.66 = 5.88$. Arrival at c_3 : $4.47 + 7.07 = 11.54 > 6$. Infeasible.

Step 2a (cont.): insert c_5 : $d(D, c_5) = 3.61$, $d(c_5, c_3) = \sqrt{4 + 4} = 2.83$. Arrival at c_3 : $3.61 + 2.83 = 6.44 > 6$. Infeasible.

$\Rightarrow$ Route 1 is $\rho_1 = (D, c_3, D)$. Open new route ρ_2 . Seed: farthest remaining from depot is c_1 or c_4 (both 4.24): pick c_1 . Continue inserting until all customers are routed.

Neighbourhood Structures for Local Search I

Local search improves a feasible VRPTW solution by applying **neighbourhood moves**: small modifications that transform one solution into another. The most effective moves operate either *within* a single route (intra-route) or *between* two routes (inter-route).

2-opt (intra-route). Given a route $(\dots, a, b, c, d, \dots)$, reverse the sub-sequence between b and c , yielding $(\dots, a, c, b, d, \dots)$. This removes two arcs and inserts two new ones. The classical 2-opt checks all $O(n^2)$ pairs per route. For VRPTW, a move is accepted only if the reversed segment is time-window feasible.

Or-Opt (intra- and inter-route). Relocate a *segment* of 1, 2, or 3 consecutive customers from its current position to a new position, either in the same route or a different route. For a segment of length 1 (single customer), Or-Opt is equivalent to the *relocate* move. The implementation checks all $O(n^2)$ possible (segment, position) pairs.

Neighbourhood Structures for Local Search II

the second category (as in Ahuja et al. [1]).

5.4.1.1 • Traditional Neighborhoods

Traditional neighborhoods can be divided into two categories: *intra-route* and *inter-route*. The neighborhoods in the former category contain solutions in which a single route is changed with respect to the reference solution, whereas, in the second category, they contain solutions obtained by moving customers between two or more routes. A thorough survey of VRPTW neighborhoods can be found in Bräysy and Gendreau [16]. Below we review the traditional neighborhoods that have been used in the most important meta-heuristics of the past decade.

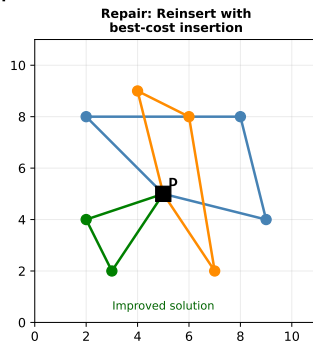
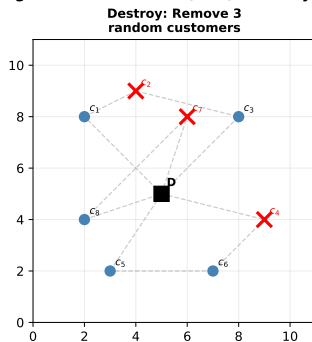
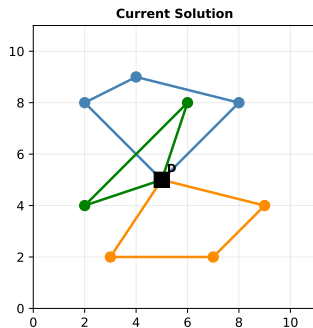
2-opt Neighborhood (intra-route). A 2-opt neighborhood (see Figure 5.1) contains solutions obtained by removing two arcs from a route and replacing them by two other arcs to reconnect the route, while changing the orientation of the subpath that does not contain the depot. In the figure the square represents the depot and the circles are customers. The dashed arcs correspond to subpaths in network G (see Section 5.2), involving one or several arcs, while each solid arc corresponds to a single arc in G . Notice that changing the orientation of the subpath $(i + 1, \dots, j + 1)$ can be problematic because of the time windows.



Large Neighbourhood Search (LNS) I

Large Neighbourhood Search was introduced by Shaw (1998) and adapted to the VRPTW. Rather than making tiny moves (like Or-Opt), LNS makes *large* moves by (1) **destroying** a significant portion of the current solution (removing q customers) and (2) **repairing** it by reinserting the removed customers.

Large Neighbourhood Search (LNS): Destroy & Repair



Large Neighbourhood Search (LNS) II

The power of LNS lies in the repair step: the q removed customers are reinserted *optimally* (or near-optimally) via an insertion heuristic, which can find combinations not reachable by small moves.

Large Neighbourhood Search (LNS) III

Basic LNS Algorithm for VRPTW

Input: initial feasible solution s , number of removals q , iterations l_{max} .

- 1 $s^* \leftarrow s$ (best solution).
- 2 **For** $i = 1$ to l_{max} :
 - a) **Destroy:** randomly select q customers from s and remove them (several destroy heuristics: random, related, worst).
 - b) **Repair:** reinsert the q customers using cheapest-insertion or regret-based insertion.
 - c) If the repaired solution s' is feasible and $\text{cost}(s') < \text{cost}(s^*)$: $s^* \leftarrow s'$.
 - d) Accept s' as new s based on acceptance criterion (e.g., always if $\text{cost}(s') \leq \text{cost}(s)$, or Metropolis criterion for ALNS).
- 3 Return s^* .

Destroy heuristics. Three common strategies:

- **Random removal:** remove q customers uniformly at random.

Large Neighbourhood Search (LNS) IV

- **Related removal:** remove q customers that are geographically close or have similar time windows (exploits correlation structure).
- **Worst removal:** remove the q customers contributing most to the objective.

Adaptive LNS (ALNS). Pisinger and Ropke (2007) extend LNS by maintaining multiple destroy/repair operators and dynamically adjusting their selection probabilities based on past performance—a *roulette-wheel* selection scheme updated by a *reward score* ρ_k for each operator k .

Large Neighbourhood Search (LNS) V

between areas of high-quality solutions. On the other hand, it adds complexity to the algorithm: evaluation of the objective may be more difficult, penalty parameters must be adjusted (potentially in an adaptive way), and one must ensure that feasible solutions are visited at least occasionally. Despite these drawbacks, allowing infeasible solutions appears to be an important tool for finding high-quality solutions.

In the current metaheuristics three types of infeasibilities are allowed: time window, vehicle capacity, and customer service violations. The last type occurs when some customers are not visited. It is the most common type in the algorithms that minimize the number of vehicles used (see the next subsection). Time window and capacity violations are typically allowed together and lead to a penalized objective function $\bar{c}(s)$:

$$\bar{c}(s) = c(s) + \alpha q(s) + \beta w(s),$$

where $c(s)$ is the standard objective function and $q(s)$ (resp., $w(s)$) measures the total capacity (resp., time-window) violation over all routes. α and β are parameters that are usually adjusted during the search, reacting to the performance of the algorithm. A good example of how the α and β parameters can be managed is given in Cordeau, Laporte, and Mercier [23].

Time-window violation along a route $r = (v_0 = 0, v_1, v_2, \dots, v_k, v_{k+1} = n + 1)$ has traditionally been calculated straightforwardly: the start of service time T_{v_i} at vertex v_i is computed as

$$T_{v_i} = \begin{cases} a_0 & \text{if } i = 0, \\ \max\{a_{v_i}, T_{v_{i-1}} + s_{v_{i-1}} + t_{v_{i-1}v_i}\} & \text{otherwise,} \end{cases}$$

Metaheuristics: Overview and Tabu Search I

Metaheuristics are algorithmic frameworks that guide neighbourhood search beyond local optima. For VRPTW, the dominant metaheuristics are tabu search (TS), simulated annealing (SA), iterated local search (ILS), and evolutionary algorithms (EA).

Why do we need metaheuristics? Gradient-free problems like VRPTW have many local optima. A simple descent that always accepts improving moves will get stuck. Metaheuristics escape local optima by temporarily accepting worsening moves, guided by memory structures, probabilistic acceptance, or population diversity.

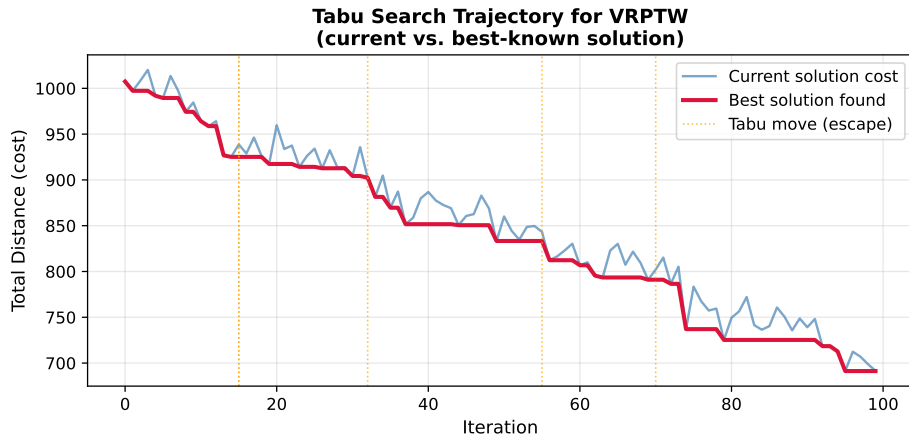
Tabu Search (TS). The word “tabu” (from the Tongan language, meaning “sacred” or “forbidden”) captures the key idea: once a move is made, it is *forbidden* (*tabu*) for the next τ iterations to prevent cycling back to the same solution. This forces the search to explore new regions of the solution space.

Tabu Search for VRPTW

Input: initial solution s , tabu tenure τ , max iterations T .

- 1 $s^* \leftarrow s$, TabuList $\leftarrow \emptyset$.
- 2 **For** $t = 1$ to T :
 - a Generate neighbourhood $\mathcal{N}(s)$ (all Or-Opt / 2-opt* moves).
 - b Find $s' = \arg \min_{s'' \in \mathcal{N}(s) \setminus \text{TabuList}} \text{cost}(s'')$ (best non-tabu move).
 - c If $\text{cost}(s') < \text{cost}(s^*)$ (aspiration): accept even if tabu; update s^* .
 - d Add the inverse of the move to TabuList for τ iterations.
 - e $s \leftarrow s'$.
- 3 **Return** s^* .

Metaheuristics: Overview and Tabu Search III



Tabu tenure τ . Choosing τ is critical:

Metaheuristics: Overview and Tabu Search IV

- Too small τ : cycling risk—the search revisits recently seen solutions.
- Too large τ : the search is too restricted—good moves are forbidden for too long.

A common heuristic sets τ randomly in $[\tau_{\min}, \tau_{\max}]$ at each iteration or adapts it based on solution quality diversity.

Notable TS implementations for VRPTW: Potvin and Rousseau (1993) introduced TS for VRPTW using Or-Opt moves. Taillard et al. (1997) proposed an adaptive memory procedure with parallel search using multiple TS threads, sharing a best-known solution. Cordeau and Laporte (2005) designed a unified TS for multiple VRP variants (VRPTW, MDVRP, PVRP) using a single penalty-based neighbourhood.

Simulated Annealing for VRPTW I

Simulated Annealing (SA), introduced by Kirkpatrick et al. (1983), is inspired by the physical process of slow cooling of a metal. When a metal cools slowly, atoms settle into a low-energy (ordered) state. Fast cooling traps atoms in high-energy (disordered) states. The analogy: the solution cost is “energy”, and the temperature T controls the probability of accepting worse solutions.

Metropolis Acceptance Criterion

Given current solution cost $f(s)$ and candidate cost $f(s')$:

$$\text{Accept } s' \text{ with probability} = \begin{cases} 1 & \text{if } f(s') \leq f(s), \\ e^{-(f(s')-f(s))/T} & \text{if } f(s') > f(s). \end{cases}$$

At high temperature T , almost all moves are accepted (exploration). As $T \rightarrow 0$, only improving moves are accepted (exploitation).

Cooling schedule. A *geometric* cooling schedule $T_{k+1} = \alpha \cdot T_k$ with $\alpha \in (0.95, 0.999)$ is standard. The initial temperature T_0 is calibrated so that an average worsening move of size Δ_0 is accepted with probability 0.5: $T_0 = -\Delta_0 / \ln(0.5)$.

Van Breedam (1994) and Czech and Czarnas (2002) applied SA to the VRPTW with good results. SA is simpler to implement than TS but typically requires more iterations to converge.

Evolutionary Algorithms for VRPTW I

Evolutionary Algorithms (EAs) maintain a **population** of solutions and evolve it through crossover and mutation, mimicking biological evolution. The diversity maintained by the population allows exploration of multiple regions of the search space simultaneously.

Genetic Algorithm (GA) for VRPTW. A chromosome encodes a permutation of all customers (ignoring route boundaries). A separate decoder translates a permutation into a feasible VRPTW solution by applying a construction heuristic (e.g., Solomon's I1) to customers in the permuted order. The fitness of a chromosome is the cost of the decoded solution.

Crossover for permutation representation. Standard crossover operators (OX, EAX, PMX) preserve partial order:

- **Order Crossover (OX):** copy a random segment from parent 1; fill remaining positions with customers from parent 2 in their original relative order.
- **Edge Assembly Crossover (EAX):** build a child by assembling “edges” (arcs) from both parents, preserving route structure.

Memetic Algorithm (MA). The most competitive EAs for VRPTW combine crossover with local search applied to each offspring—a **memetic** approach. The Nagata-Bräysy (NB09) and Vidal et al. (BC12, VCGP13) algorithms are leading examples.

Vidal et al. Hybrid Genetic Search (VCGP13)

- 1 Initialise population of P solutions (using I1 heuristic).
- 2 **Repeat until stopping criterion:**
 - a) Select two parents p_1, p_2 by tournament selection.
 - b) Apply OX+split crossover: create offspring o by OX crossover, then apply the “split” procedure to optimally partition o into routes.
 - c) Apply intensive local search (Or-Opt, 2-opt*) to o .
 - d) Add o to population; if population overflows, remove the most similar (least diverse) individual with worst fitness.
- 3 Return best solution found.

The **Split procedure** by Prins (2004) decomposes a giant tour (a permutation of all customers, ignoring depot) into m routes optimally using dynamic programming in $O(nm)$ time. This allows crossover operators to work on permutations without worrying about route boundaries.

Population-Based Search and Path Relinking I

Path Relinking (PR) is an intensification strategy that generates new solutions by exploring *paths* between pairs of elite solutions in the solution space. Given two solutions s_{init} and s_{guide} , PR applies moves that transform s_{init} into s_{guide} step by step, evaluating each intermediate solution.

Guided Local Search (GLS) augments the objective function with penalty terms on solution features (e.g., arcs) that have appeared frequently in previous local optima but have not led to good solutions. This allows GLS to escape from local optima by making “costly” features expensive.

Iterated Local Search (ILS). ILS alternates between a perturbation step (to escape a local optimum) and a local search step:

- 1 Apply local search to obtain local optimum s^* .
- 2 **Repeat:** perturb s^* (e.g., remove and reinsert a random sequence of customers), apply local search to obtain s' , accept s' based on a criterion (e.g., s' better than s^*).

ILS for VRPTW (Irnich et al. 2010) achieves near-optimal results on Solomon benchmarks with very short running times.

Python Implementation: Savings Algorithm I

The **Clarke-Wright savings algorithm** provides a practical starting point for VRPTW. Although it was originally designed for CVRP, it can be extended to handle time windows by checking feasibility during the merging step.

The savings $s(i, j)$ for merging customer i and customer j into a single route is:

$$s(i, j) = d(0, i) + d(0, j) - d(i, j),$$

where $d(0, i)$ is the distance from the depot to customer i . Merging i and j saves $s(i, j)$ compared to serving them on separate routes. Pairs with the highest savings are merged first, subject to capacity and (for VRPTW) time-window feasibility.

Worked Example: Savings Calculation

Depot $D = (0, 0)$, customers: $c_1 = (3, 4)$, $c_2 = (6, 2)$, $c_3 = (5, 6)$, $c_4 = (1, 5)$.

$$d(D, c_1) = 5.00, \quad d(D, c_2) = 6.32, \quad d(D, c_3) = 7.81, \quad d(D, c_4) = 5.10.$$

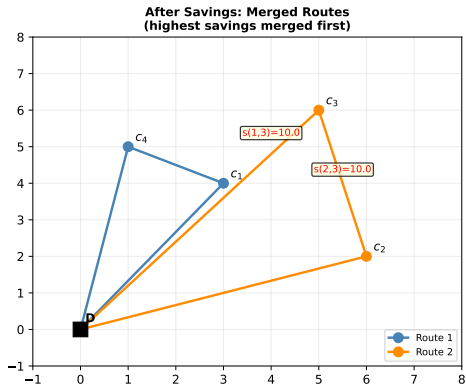
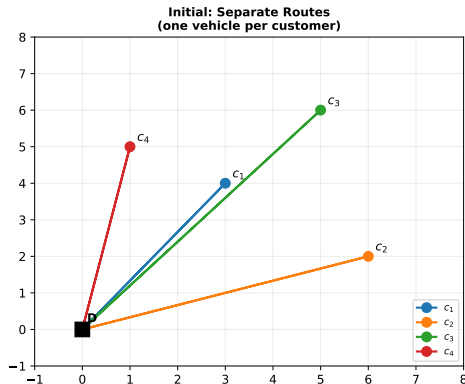
$$s(1, 2) = 5.00 + 6.32 - d(c_1, c_2) = 11.32 - \sqrt{9 + 4} = 11.32 - 3.61 = 7.71.$$

$$s(1, 3) = 5.00 + 7.81 - \sqrt{4 + 4} = 12.81 - 2.83 = 9.98 \quad (\text{highest!})$$

Merge c_1 and c_3 first (subject to time-window and capacity feasibility).

Python Implementation: Savings Algorithm III

Clarke-Wright Savings Algorithm — Worked Example

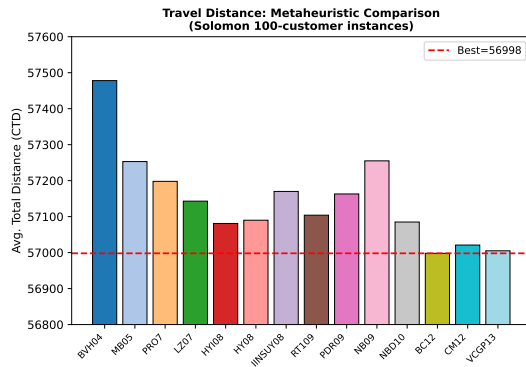
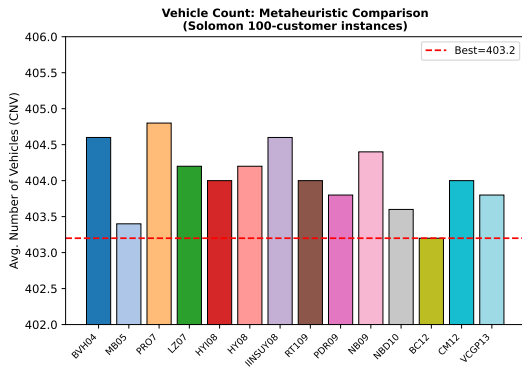


Python Code: Clarke-Wright Savings + 2-opt for VRPTW

```
1 import math, itertools
2
3 def euclidean(a, b):
4     return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)
5
6 def compute_savings(depot, customers):
7     """Return sorted list of (saving, i, j) pairs."""
8     savings = []
9     for i, ci in customers.items():
10         for j, cj in customers.items():
11             if i >= j: continue
12             s = euclidean(depot, ci) + euclidean(depot, cj) \
13                 - euclidean(ci, cj)
14             savings.append((s, i, j))
15     savings.sort(reverse=True)
16     return savings
17
18 def route_feasible(route, customers, depot, tw, demand, Q):
19     """Check capacity and time-window feasibility."""
20     load = sum(demand[c] for c in route)
21     if load > Q: return False
22     t = 0.0
23     prev = depot
24     for c in route:
25         t += euclidean(prev, customers[c])
26         if t > tw[c][1]: return False # late arrival
27         t = max(t, tw[c][0]) + 1.0 # wait + service time=1
```

Metaheuristic Comparison on Solomon Benchmarks I

VRPTW Metaheuristic Performance on Solomon Benchmarks



Metaheuristic Comparison on Solomon Benchmarks II

The table below summarises the best-known results on Solomon's 100-customer instances (averaged over all instances in each class), based on the metaheuristic comparison in Table 5.2 of the book. CNV = cumulative number of vehicles; CTD = cumulative total distance.

Algorithm	Year	Avg. CNV	Avg. CTD
Potvin-Bengio (PRO7)	2007	404.8	57,198
Liu-Jiang (LZ07)	2007	404.2	57,143
HYI08	2008	404.0	57,081
Irnich et al. (RT109)	2009	404.0	57,104
Nagata-Bräysy (NB09)	2009	404.4	57,255
Nagata et al. (NBD10)	2010	403.6	57,085
BC12	2012	403.2	56,998
Vidal et al. (VCGP13)	2013	403.8	57,005

Metaheuristic Comparison on Solomon Benchmarks III

The dominant metaheuristics (BC12, VCGP13) combine genetic crossover with intensive local search. The primary gains in recent years have come from fleet minimisation (lower CNV) rather than distance reduction; once the fleet is minimised, improving distance requires fine-grained local search.

Extension: Split Deliveries (SDVRPTW) I

In the standard VRPTW, each customer is visited exactly once. In the **Split Delivery VRP with Time Windows (SDVRPTW)**, a customer with large demand (exceeding Q) may be served by *multiple* vehicles, each delivering a portion of the demand.

Why split? Split deliveries can reduce the total distance and fleet size when customer demands are large relative to vehicle capacity. For example, a warehouse requiring 1.5 units of capacity can be served by one vehicle delivering 1.0 and another delivering 0.5, avoiding the need for a dedicated third vehicle.

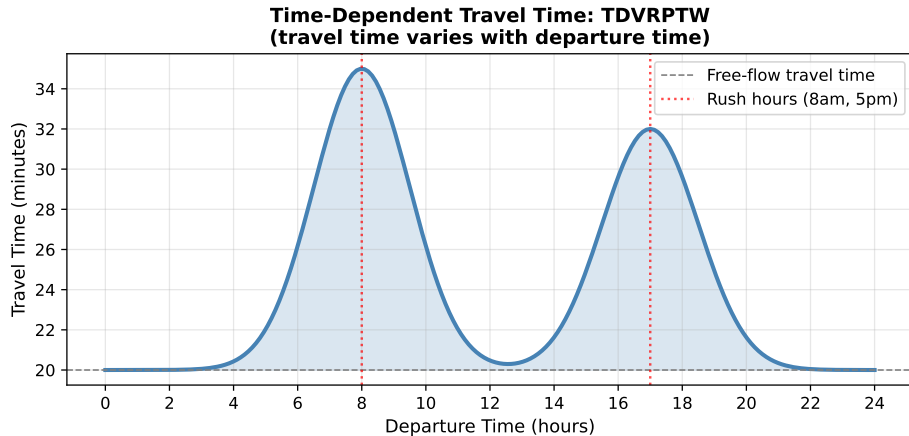
Formulation change. In Model I, constraint (5.2) is relaxed from $\sum_{k,j} x_{ijk} = 1$ to $\sum_{k,j} x_{ijk} \geq 1$. A new continuous variable $d_{ik} \geq 0$ records the quantity delivered to customer i by vehicle k , and the capacity constraint becomes $\sum_i d_{ik} \leq Q$ for each k . An additional constraint $\sum_k d_{ik} = q_i$ ensures total demand is met.

Split deliveries substantially increase the solution space but also the solution quality. Heuristics for SDVRPTW extend Solomon's insertion by allowing partial insertions.

Extension: Time-Dependent Travel Times (TDVRPTW) I

In urban logistics, travel times between locations vary throughout the day due to congestion. The **Time-Dependent VRPTW (TDVRPTW)** replaces the fixed travel time t_{ij} with a function $t_{ij}(d)$ of the departure time d from node i .

Extension: Time-Dependent Travel Times (TDVRPTW) II



Extension: Time-Dependent Travel Times (TDVRPTW) III

FIFO property. For the SPPRC labelling algorithm to remain valid, the resource extension function must be *monotone*: arriving later cannot help you arrive earlier. This requires $t_{ij}(d)$ to satisfy the *FIFO* (First-In, First-Out) property:

$$d_1 < d_2 \implies d_1 + t_{ij}(d_1) < d_2 + t_{ij}(d_2),$$

meaning that a vehicle leaving later always arrives later. Piecewise linear functions satisfying FIFO are the standard modelling choice, and they arise naturally from historical speed data.

Impact on algorithms. The label-setting algorithm for SPPRC extends directly to TDVRPTW by replacing $T_j = \max(e_j, T_i + s_i + t_{ij})$ with $T_j = \max(e_j, T_i + s_i + t_{ij}(T_i + s_i))$. All other feasibility checks remain the same. Algorithms by Figliozzi (2012) and Gendreau et al. (2015) handle TDVRPTW heuristically; exact methods remain limited to small instances.

Extension: Multi-Depot VRPTW (MDVRPTW) I

In the **Multi-Depot VRPTW (MDVRPTW)**, vehicles are stationed at multiple depots rather than a single central depot. Each vehicle departs from its home depot, serves a set of customers, and returns to the same (or any) depot.

Formulation. Extend Model I by indexing depots $d \in D$. Each vehicle k is assigned to depot $d(k)$. The flow conservation constraints (5.3)–(5.5) are modified to apply to the correct depot. The MDVRPTW generalises the VRP with multiple depots and inherits all the complexity of the VRPTW.

Customer assignment. A key sub-problem in MDVRPTW is *which depot should serve which customers*. This adds a combinatorial layer on top of routing. Cordeau et al. (2001) proposed a tabu search for MDVRPTW that assigns customers to depots dynamically.

Heterogeneous fleets. In the **Heterogeneous Fleet VRPTW (HFVRPTW)**, vehicles differ in capacity Q_k , speed, and cost. Assigning the right vehicle type to each route is an additional optimisation dimension. Mixed fleets arise in city logistics (small vans for tight streets, large trucks for bulk deliveries).

Extension: Stochastic VRPTW I

In the **Stochastic VRPTW (SVRPTW)**, some problem data are uncertain: customer demands, travel times, or even whether a customer is present may be random. Two approaches handle stochasticity:

1. Stochastic programming. Model uncertainty explicitly with probability distributions. Formulate a two-stage problem:

- **Stage 1 (here-and-now):** determine routes before realisation of random variables.
- **Stage 2 (wait-and-see):** recourse actions after realisation (e.g., re-dispatching a vehicle if a customer's demand exceeds capacity).

The objective minimises expected total cost (routing + recourse).

2. Robust optimisation. Find routes that are feasible for the *worst case* realisations of uncertainty within a defined uncertainty set $\mathcal{U}$ (e.g., travel times vary within $\pm 20\%$ of nominal). Robust routes trade higher average cost for immunity to worst-case disruptions.

Dynamic VRPTW. Customer requests arrive in real time during the planning horizon. The dispatcher must insert new requests into existing routes while vehicles are en route. Online algorithms and re-optimisation heuristics (e.g., rescheduling after each new arrival) address this setting. Larsen (1999) introduced the degree of dynamism to measure how online a VRPTW instance is.

Extension: Driver Considerations I

Real-world VRPTW instances must account for **driver regulations**:

- **Maximum driving time:** EU regulations limit continuous driving to 4.5 hours before a mandatory 45-minute break; total daily driving is capped at 9 hours.
- **Shift constraints:** drivers may only work within specified shifts (e.g., 06:00–18:00), modelled as an additional time window at the depot.
- **Multiple time windows:** some customers accept delivery in two separate time intervals during the day (e.g., [08 : 00, 10 : 00] or [14 : 00, 16 : 00]), arising from receiving dock schedules. This is the **VRPTW with multiple time windows**, which requires tracking which window is used for each customer.

Modelling multiple time windows. For customer i with windows $[e_i^1, l_i^1], [e_i^2, l_i^2], \dots$, the time variable T_i must satisfy $T_i \in [e_i^1, l_i^1] \cup [e_i^2, l_i^2] \cup \dots$. This non-convex set complicates the SPPRC: the label algorithm must track which window the vehicle targets, potentially branching at each customer.

Conclusions I

This chapter has surveyed the Vehicle Routing Problem with Time Windows (VRPTW) from mathematical foundations to state-of-the-art algorithms. The following main points summarise the chapter:

Mathematical formulations. The VRPTW can be modelled as a three-index arc-flow MIP (Model I) or a set-partitioning problem (Model II). Model II gives tighter LP bounds and is the basis of the best exact algorithms. Hard and soft time windows, service times, and waiting times all fit naturally within both frameworks.

Exact methods. Branch-and-price, driven by column generation, is the dominant exact approach. The pricing subproblem (ESPPRC via label-setting) is the computational bottleneck. All 56 Solomon 100-customer instances are solved to optimality; larger instances remain open. Recent advances (ng-relaxation, subset-row cuts, bidirectional labelling) continue to push the frontier.

Heuristics and metaheuristics. Solomon's I1 insertion provides fast initial solutions. Local search with Or-Opt, 2-opt*, and cross-exchange moves improves them. Metaheuristics (tabu search, LNS, hybrid genetic search) achieve near-optimal solutions on benchmark instances in seconds. The best algorithms

Conclusions II

(BC12, VCGP13) combine genetic crossover with intensive local search and achieve cumulative vehicle counts within 0.1% of the best known on Solomon benchmarks.

Extensions. The VRPTW framework has been extended to handle split deliveries, time-dependent travel times, multiple depots, heterogeneous fleets, stochastic demands, dynamic requests, multiple time windows, and driver regulations. Most extensions can be handled by modifying the SPPRC pricing step.

Future Research Directions I

Based on the state of the art surveyed in this chapter, the following research directions are most promising:

- 1. Scaling exact methods.** Current exact algorithms solve Solomon's 100-customer instances but not the Gehring-Homberger 200- and 400-customer instances. Advances in ng-route relaxation, stronger cuts (multirow, packing inequalities), and parallel branch-and-price trees are needed.
- 2. Machine learning integration.** Learning to predict which routes will appear in optimal solutions (*warm starting* column generation) or to tune metaheuristic hyperparameters adaptively could substantially speed up both exact and heuristic methods. Recent work on attention-based neural combinatorial optimisation (Kool et al. 2019) directly generates VRPTW solutions but still lags behind dedicated metaheuristics on quality.
- 3. Real-time and online VRPTW.** As e-commerce grows, same-day delivery creates highly dynamic instances where orders arrive throughout the day. Algorithms must balance re-optimisation quality against re-scheduling disruption (drivers cannot be recalled from mid-route).

Future Research Directions II

4. Green and electric VRP. Electric vehicles have limited range and require recharging during routes. The **Electric VRPTW (E-VRPTW)** adds a recharging resource constraint (energy level) and charging station visits to the SPPRC. This is an active research frontier driven by sustainability requirements.

5. Stochastic and robust VRPTW at scale. Two-stage stochastic programs for VRPTW with hundreds of customers and realistic uncertainty sets remain computationally out of reach. Scenario-based decomposition and distributionally robust formulations offer promising directions.

Key Takeaway

The VRPTW is one of the most thoroughly studied combinatorial optimisation problems. Its combination of rich practical structure, well-defined benchmarks, and clear algorithmic challenges has made it a proving ground for exact and heuristic methods that now influence the broader field of integer programming and metaheuristics.

Key References I



M.M. Solomon.

Algorithms for the vehicle routing and scheduling problems with time window constraints.
Operations Research, 35(2):254–265, 1987.



M. Desrochers, J. Desrosiers, and M.M. Solomon.

A new optimization algorithm for the vehicle routing problem with time windows.
Operations Research, 40(2):342–354, 1992.



R. Baldacci, A. Mingozzi, and R. Roberti.

Recent exact algorithms for solving the vehicle routing problem under capacity and time window constraints.
European Journal of Operational Research, 218(1):1–6, 2012.



J.-Y. Potvin and J.-M. Rousseau.

A parallel route building algorithm for the vehicle routing and scheduling problem with time windows.
European Journal of Operational Research, 66(3):331–340, 1993.

Key References II



É. Taillard, P. Badeau, M. Gendreau, F. Guertin, and J.-Y. Potvin.

A tabu search heuristic for the vehicle routing problem with soft time windows.

Transportation Science, 31(2):170–186, 1997.



P. Shaw.

Using constraint programming and local search methods to solve vehicle routing problems.

In *Principles and Practice of Constraint Programming*, Lecture Notes in Computer Science 1520, pp. 417–431, 1998.



D. Pisinger and S. Ropke.

A general heuristic for vehicle routing problems.

Computers & Operations Research, 34(8):2403–2435, 2007.



Y. Nagata and O. Bräysy.

A powerful route minimization heuristic for the vehicle routing problem with time windows.

Operations Research Letters, 37(5):333–338, 2009.

Key References III



T. Vidal, T.G. Crainic, M. Gendreau, and C. Prins.

A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time windows.

Computers & Operations Research, 40(1):475–489, 2013.



M. Gendreau, G. Ghiani, and E. Guerriero.

Time-dependent routing problems: A review.

Computers & Operations Research, 64:189–197, 2015.



S. Irnich, P. Toth, and D. Vigo.

The family of vehicle routing problems.

Chapter 1 in *Vehicle Routing: Problems, Methods, and Applications*, MOS-SIAM Series on Optimization, 2014.



J.-F. Cordeau, M. Gendreau, and G. Laporte.

A tabu search heuristic for periodic and multi-depot vehicle routing problems.

Networks, 30(2):105–119, 1997.

Summary of VRPTW Algorithms

Method	Type	Optimal?	Scale
Branch-and-Bound (MIP)	Exact	Yes	$n \leq 25$
Column Generation	LP bound	N/A	Any n
Branch-and-Price	Exact	Yes	$n \leq 100$
B&P&C (ng-routes)	Exact	Yes	$n \leq 200$
Solomon I1	Construction	No	Any n
Or-Opt / 2-opt*	Local search	No	Any n
Tabu Search	Metaheuristic	No	Any n
Simulated Annealing	Metaheuristic	No	Any n
LNS / ALNS	Metaheuristic	No	Any n
Hybrid Genetic Search	Metaheuristic	No	Any n

Construction heuristics provide fast initial solutions; metaheuristics refine them; exact methods verify optimality for small to medium instances.